



Laura Valentina Ramirez Farfan

Jhonny Alberto Cotrina Buitrago

Gerson Eliut Torrado Guerrero

Facultad Ingeniería Sistemas, Corporación Unificada Nacional de Educación Superior

Código 57194: Programación Avanzada

Docente, Juan Esteban Caicedo Palacio

12 de enero de 2025

[Link del Video Explicativo](#)

[Link de la Carpeta Drive que contiene este trabajo, el vídeo y el código entero.](#)

## Índice

|       |                                                         |    |
|-------|---------------------------------------------------------|----|
| 1     | Introducción .....                                      | 5  |
| 2     | Objetivos .....                                         | 6  |
| 2.1   | Objetivo General .....                                  | 6  |
| 2.2   | Objetivos Específicos .....                             | 6  |
| 3     | Resumen .....                                           | 6  |
| 4     | Desarrollo .....                                        | 8  |
| 4.1.1 | Apache Maven .....                                      | 8  |
| 4.1.2 | JPA Controller .....                                    | 8  |
| 4.2   | Estructura modular .....                                | 9  |
| 4.2.1 | Ventajas de la Estructura Modular en Barberiacjev ..... | 9  |
| 4.2.2 | Separación de Responsabilidades .....                   | 10 |
| 4.3   | Diseño de la App .....                                  | 12 |
| 4.3.1 | Ventana Principal .....                                 | 12 |
| 4.3.2 | Ventana de Registrar Datos .....                        | 13 |
| 4.3.3 | Ventana Ver Datos .....                                 | 15 |
| 4.3.4 | Función Editar datos .....                              | 16 |
| 4.4   | Clases Principales (Código) .....                       | 17 |
| 4.4.1 | Clase Cliente .....                                     | 17 |
| 4.4.2 | Clase Servicio .....                                    | 20 |
| 4.4.3 | Clase Controladora .....                                | 22 |
| 4.5   | Interfaz de Usuario (Código) .....                      | 23 |

|       |                           |    |
|-------|---------------------------|----|
| 4.5.1 | Panel Principal .....     | 23 |
| 4.5.2 | Ingreso de Datos .....    | 24 |
| 4.5.3 | Edición de Datos .....    | 26 |
| 4.5.4 | Consulta de Datos .....   | 30 |
| 5     | Conclusiones .....        | 32 |
| 6     | Aspecto s a mejorar ..... | 33 |
| 7     | Referencias.....          | 34 |

El contenido del presente documento es de nuestra autoría y es un reflejo de nuestro trabajo personal y manifestamos que, ante cualquier notificación de plagio, copia o falta a la fuente original, somos responsables directamente en lo legal, económico y administrativo sin afectar al director del trabajo, a la universidad y a cuantas instituciones han colaborado en dicho trabajo, asumiendo las consecuencias derivadas de tales prácticas.



---

Jhonny Alberto Cotrina Buitrago



---

Laura Valentina Ramirez Farfan



---

Gerson Eliut Torrado Guerrero

## 1 Introducción

Este proyecto tiene como objetivo principal el desarrollo de una aplicación en Java que optimice la gestión de clientes y servicios en la barbería "**Barberiacjev**". A través del uso de Maven para la gestión de dependencias y **JPA Controller** para la conexión y manejo eficiente de la base de datos alojada en **Wampserver64**, se busca mejorar la organización interna y la atención al cliente.

El desarrollo incluyó la creación de una base de datos funcional con tablas para clientes, servicios y sucursales. Además, se implementó una estructura modular en el código fuente que incorpora clases principales como:

- **Cliente:** Maneja los datos personales y preferencias de los usuarios.
- **Servicio:** Representa las opciones ofrecidas por la barbería, como descripciones y precios.
- **Controladora:** Administra las operaciones entre la interfaz de usuario y la base de datos mediante **JPA Controller**.

Se diseñaron módulos específicos para la interfaz gráfica, incluyendo un panel principal, ingreso, edición y consulta de datos. Esto permitió una interacción intuitiva y eficiente tanto para los usuarios como para el personal administrativo.

La aplicación fue probada rigurosamente, asegurando su correcto funcionamiento bajo diferentes escenarios. Este desarrollo tecnológico brinda a la barbería una solución moderna para la administración de sus procesos, mejorando tanto la experiencia del cliente como la operatividad de la empresa.

## 2 Objetivos

### 2.1 Objetivo General

Desarrollar una aplicación en Java utilizando **Maven** y **JPA Controller** que permita a la barbería "**Barberiacjev**" gestionar de manera eficiente el registro de clientes y la administración de servicios, mejorando la organización y la calidad de atención tanto para los clientes como para los empleados.

### 2.2 Objetivos Específicos

- Diseñar y desarrollar una base de datos funcional utilizando **Wampserver64** para almacenar información sobre clientes, servicios y sucursales de la barbería.
- Implementar una interfaz de usuario intuitiva en la aplicación que permita a los clientes registrar sus datos personales, seleccionar servicios y elegir la sede de su preferencia.
- Integrar funcionalidades de gestión para que la administración pueda acceder a informes y supervisar la atención al cliente y el desempeño del personal.
- Garantizar la conexión eficiente y segura entre la aplicación y la base de datos mediante el uso de **JPA Controller**.
- Validar y probar la aplicación para asegurar su correcto funcionamiento en diferentes escenarios de uso.

## 3 Resumen

El proyecto tiene como objetivo solucionar una problemática común en muchas barberías: la falta de una gestión eficiente para el registro de clientes y la administración de los servicios ofrecidos. La barbería "**Barberiacjev**" busca implementar una aplicación moderna que permita registrar los datos de identificación de los clientes, así como el servicio de corte elegido y la sede preferida. Para ello, se desarrolló una aplicación en Java utilizando el framework Maven, con la integración de JPA Controller para la gestión de datos y una base de

datos alojada en **Wampserver64**. El resultado final es una herramienta que mejora la organización interna de la barbería y facilita la experiencia del cliente.

## 4 Desarrollo

El desarrollo de la aplicación comenzó con la planificación y diseño de la base de datos, la cual incluye tablas para clientes, servicios y sucursales. Posteriormente, se configuró el entorno de desarrollo utilizando Maven para la gestión de dependencias y Wampserver64 como servidor de base de datos.

### 4.1.1 *Apache Maven*

Es una herramienta de gestión de proyectos y construcción automatizada utilizada principalmente en proyectos Java. Su propósito es simplificar la configuración, compilación y despliegue de proyectos al proporcionar un modelo estructurado y herramientas para manejar dependencias externas (bibliotecas y frameworks) de manera eficiente.

- **Gestión de dependencias:** Maven utiliza un archivo llamado **pom.xml (Project Object Model)** para declarar las dependencias, que se descargan automáticamente desde repositorios en línea. Esto elimina la necesidad de manejar manualmente los archivos de las bibliotecas.
- **Estructura estándar:** Proporciona una estructura predeterminada para los proyectos, lo que facilita su organización y mantenimiento.
- **Plugins:** Maven incluye una amplia variedad de complementos que automatizan tareas como compilación, pruebas, empaquetado y despliegue.

### 4.1.2 *JPA Controller*

Los **JPA Controllers** son una implementación común en el marco de **Java Persistence API (JPA)**, que se utiliza para interactuar con bases de datos relacionales de manera sencilla y orientada a objetos.



- **JPA:** Es una especificación de Java que define cómo se deben mapear las clases de Java a tablas de una base de datos y viceversa. Simplifica el acceso a datos al abstraer las consultas SQL mediante el uso de entidades Java.
- **Controllers:** Son clases generadas o implementadas para manejar operaciones **CRUD** (Crear, Leer, Actualizar, Eliminar) sobre las entidades definidas. Actúan como intermediarios entre la base de datos y la lógica de negocio.
- **Ventajas:** Los **JPA Controllers** automatizan tareas complejas como transacciones, validaciones y la persistencia de datos, asegurando un manejo seguro y eficiente.

Se implementó una estructura modular en el código fuente para garantizar la separación de responsabilidades y facilitar el mantenimiento.

## 4.2 Estructura modular

En nuestro proyecto es fundamental la estructura modular para garantizar un desarrollo ordenado, mantenible y escalable. Esta organización aporta múltiples beneficios:

### 4.2.1 *Ventajas de la Estructura Modular en Barberiacjev*

La implementación de una estructura modular en el proyecto **Barberiacjev** proporciona múltiples beneficios, asegurando que cada componente esté bien definido y pueda funcionar de manera eficiente e independiente. Estas ventajas son esenciales para mantener la calidad y escalabilidad de la aplicación.

### 4.2.2 Separación de Responsabilidades

Cada módulo en la aplicación de **Barberiacjev** cumple una función específica:

**Cliente:** Se encarga de gestionar los datos personales y preferencias de los clientes.

**Servicio:** Administra la información relacionada con los servicios ofrecidos por la barbería.

**Controladora:** Actúa como intermediaria entre la interfaz de usuario y la base de datos, permitiendo operaciones CRUD de manera eficiente.

Esta separación asegura que los componentes puedan trabajar de forma independiente, reduciendo la complejidad del sistema y facilitando su mantenimiento.

#### Reutilización de Código

Los módulos bien diseñados son reutilizables dentro del mismo proyecto o en otros proyectos futuros. Por ejemplo, la clase Controladora podría extenderse fácilmente para manejar nuevas entidades o adaptarse a diferentes escenarios, como la gestión de promociones o reservas.

#### Mantenibilidad

Si surge un problema o es necesario implementar una nueva funcionalidad, es sencillo identificar y modificar únicamente el módulo afectado. Por ejemplo, si se desea agregar un nuevo atributo a los clientes, solo será necesario actualizar la clase Cliente y los módulos que interactúan con ella, sin impactar otros componentes.

#### Escalabilidad

La estructura modular facilita la adición de nuevas funcionalidades sin alterar los módulos ya existentes. En el caso de Barberiacjev, podrías integrar módulos adicionales para gestionar

membresías, historial de citas o estadísticas de uso, asegurando que la base del sistema permanezca intacta.

### **Facilita las Pruebas**

Dado que cada módulo opera de manera independiente, es posible realizar pruebas unitarias específicas para cada uno. Esto permite identificar problemas rápidamente y garantizar la calidad de cada componente antes de integrarlos en el sistema completo.

### **Colaboración en Equipo**

En un escenario de desarrollo colaborativo, la estructura modular permite que diferentes miembros del equipo trabajen simultáneamente en módulos distintos sin interferencias. Por ejemplo, un desarrollador podría trabajar en la clase Cliente mientras otro se ocupa de mejorar la interfaz de usuario.

### **Claridad y Organización**

Dividir el proyecto en módulos lógicos mejora la legibilidad del código y facilita la incorporación de nuevos desarrolladores al equipo. Estos podrán entender fácilmente cómo están estructuradas las funcionalidades clave y adaptarse rápidamente al flujo del proyecto.

### **Aplicación en Barberiacjev**

En nuestro proyecto de Barberiacjev, la estructura modular garantiza que las principales funciones, como la gestión de clientes, servicios y la conexión con la base de datos, estén organizadas y sean claras. **Por ejemplo:**

Si se necesita modificar la forma en que se realizan las transacciones en la base de datos, solo será necesario actualizar la clase Controladora, sin afectar los módulos de Cliente o Servicio.

En caso de querer añadir una funcionalidad como el historial de citas, se podría implementar un nuevo módulo sin alterar los existentes, asegurando un desarrollo continuo y ordenado.

### 4.3 Diseño de la App

#### 4.3.1 Ventana Principal

En la ventana principal nos inspiramos en los colores marrones semejantes al cuero ya que son del color de los delantales que usan los barberos, además que hicimos la interfaz muy simple para el cliente esta consta de los siguientes botones implementando Swing Controls

(Button):

- *Registrar*
- *Ver Datos*
- *Salir de la app.*



#### 4.3.2 Ventana de Registrar Datos

**Ingresar datos del cliente como:** usando los Swing Controls específicamente el Text Field para los campos de texto en los que se llenaran los siguientes datos:

- *Nombre y Apellido*
- *Cedula*
- *Contacto*
- *Residencia*

**Ingresar datos del Servicio a Tomar:** usando los Swing Controls pero en este apartado se usó Text Field y Combo box para los campos de texto en los que se llenaran los siguientes datos:

- *Tipo de Cabello (Liso, Rizado, Ondulado, Afro) Combo box*
- *Que tipo de Corte deseas hacerte*
- *Sede (Chapinero, Candelaria, Suba, Bosa) Combo box*
- *Barbero (Valentina, Cesar, Jhonny, Eliut) Combo box*
- *Adicionales (Corte de Cejas, Corte de barba, Tinturado) Combo box*
- *Deseas Asesoría (Si, No) Combo Box*

Además, se implementaron Swing Controls (**Button**) para Limpiar y Guardar.



A screenshot of a web application for a barber shop named "BARERIA CJEV". The background features a wooden surface with various barber tools: a red electric shaver, silver scissors, a straight razor, a wooden comb, and a shaving brush. A yellow circular logo with a stylized 'B' is centered at the top. The application form is dark-themed and contains two main sections: "Datos Cliente" and "Datos Servicio".

**Datos Cliente**

Nombre y Apellido

Cedula

Contacto

Residencia

**Datos Servicio**

Tipo de Cabello

Que tipo de corte Deseas Hacerte

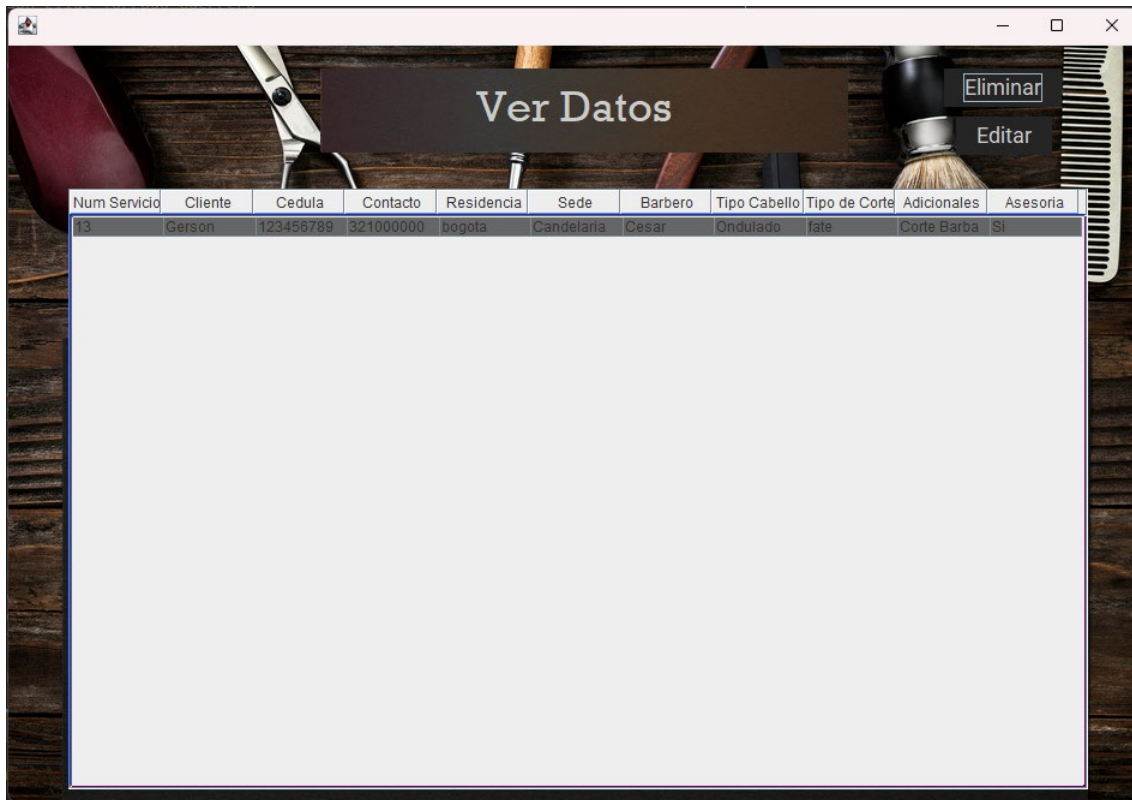
Sede  Barbero

Adicionales  Deseas Asesoria

**Limpiar** **Guardar**

#### 4.3.3 Ventana Ver Datos

En esta ventana se implemento un formulario para ver los clientes y los servicios que tomaron cada uno de los usuarios. Además, que se incluye Swing Controls (**Button**) que tienen como función eliminar o editar esta tabla que se va generando a medida que los clientes vayan registrando sus datos.



#### 4.3.4 Función Editar datos

Al dar click en el Swing Controls (**Button**) **Editar** este nos desplegara otra ventana donde podremos apreciar una ventana similar a la de registrar datos, pero esta va a tener ya los datos de la fila que hallamos escogido en la tabla que esta en la ventana ver datos, donde desde allí modificar los datos, (cabe resaltar que esta edición también se verá reflejada en la base de Datos.)

### Editar Datos Cliente

Nombre y Apellido  
Gerson

Cedula  
123456789

Contacto  
321000000

Residencia  
bogota

**Limpiar**

### Editar Datos Servicio

Tipo de Cabello Ondulado

Que tipo de corte Deseas Hacerte  
rate

Sede Candelaria Barbero Cesar

Adicionales Corte Barba Deseas Asesoría Si

**Guardar Cambios**

Como Aclaración el código se comentó luego de hacerse el video, por ese motivo en el video no aparece bien comentado la estructura del código y sus funciones, en algunas partes de decidí con ayuda de IA comentar de forma más técnica nuestro código, pero en otras dejamos nuestros propios comentarios.

Además que se reorganizo la tabla que nos sirve para ver los datos ya que estaba desorganizada y dejarla como esta en la base de datos.

Tabla como lo mostramos en el video:





| Num Servicio | Sede       | Barbero   | Tipo Cabello | Tipo de Corte | Adicionales    | Asesoría | Cliente   | Cedula     | Contacto   | Residencia |
|--------------|------------|-----------|--------------|---------------|----------------|----------|-----------|------------|------------|------------|
| 1            | Chapinero  | Cesar     | Ondulado     | Lo que sea    | Corte Barba    | Si       | Jeremy    | 1110605961 | 3214816506 | Bogota     |
| 2            | Chapinero  | Valentina | Ondulado     | Despunte      | Corte de ce... | Si       | Gerson    | 1110605961 | 3214816506 | Bogota     |
| 3            | Suba       | Cesar     | Liso         | Mid fate      | Corte de ce... | Si       | Jhonny    | 123456789  | 987654321  | Funza      |
| 4            | Candelaria | Jhonny    | Ondulado     | Despuntado    | Tinturado      | Si       | Valentina | 36274      | 33333      | Pacho      |
| 9            | Candelaria | Jhonny    | Ondulado     | Despunte      | Corte Barba    | Si       | gerson    | 1110605961 | 3214816506 | Bogota     |

**Tabla con la reorganización:**



| Num Servicio | Cliente | Cedula    | Contacto  | Residencia | Sede       | Barbero | Tipo Cabello | Tipo de Corte | Adicionales | Asesoría |
|--------------|---------|-----------|-----------|------------|------------|---------|--------------|---------------|-------------|----------|
| 13           | Gerson  | 123456789 | 321000000 | bogota     | Candelaria | Cesar   | Ondulado     | fate          | Corte Barba | Si       |

#### 4.4 Clases Principales (Código)

Se crearon las siguientes clases principales:

##### 4.4.1 Clase Cliente

Esta clase define los atributos de los clientes, como nombre, identificación, contacto y preferencias. También incluye los métodos necesarios para gestionar la información de los clientes.

```

package com.mycompany.barberiacjev4.logica;
import java.io.Serializable;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
/*
 * Clase Cliente.
 * Representa a un cliente en el sistema de la barbería. Esta clase utiliza
 * anotaciones de JPA para mapear la información en la base de datos.
 */
@Entity
public class Cliente implements Serializable {
    // Identificador único para el cliente (clave primaria en la base de datos)
    @Id
    @GeneratedValue(strategy= GenerationType.SEQUENCE)
    private int id_cliente;
    private String Cedula;
    private String Num_Contacto;
    private String Nombre_Apellido;
    private String Residencia;
    /*
     * Constructor por defecto.
     * Necesario para las operaciones de JPA.
     */
    public Cliente() {
    }
    /*
     * Constructor con parámetros.
     * Permite inicializar un cliente con todos sus atributos.
     * @param id_cliente Identificador único del cliente
     * @param Cedula Número de cédula del cliente
     * @param Num_Contacto Número de contacto del cliente
     * @param Nombre_Apellido Nombre completo del cliente
     * @param Residencia Dirección de residencia del cliente
     */
    public Cliente(int id_cliente, String Cedula, String Num_Contacto, String Nombre_Apellido, String Residencia) {
        this.id_cliente = id_cliente;
        this.Cedula = Cedula;
        this.Num_Contacto = Num_Contacto;
        this.Nombre_Apellido = Nombre_Apellido;
        this.Residencia = Residencia;
    }
}

```

```
// Métodos getter y setter para cada atributo
/*
 * Obtiene el identificador único del cliente.
 *
 * @return id_cliente
 */
public int getId_cliente() {
    return id_cliente;
}
public void setId_cliente(int id_cliente) {
    this.id_cliente = id_cliente;
}
public String getCedula() {
    return Cedula;
}
public void setCedula(String Cedula) {
    this.Cedula = Cedula;
}
public String getNum_Contacto() {
    return Num_Contacto;
}
public void setNum_Contacto(String Num_Contacto) {
    this.Num_Contacto = Num_Contacto;
}
public String getNombre_Apellido() {
    return Nombre_Apellido;
}
public void setNombre_Apellido(String Nombre_Apellido) {
    this.Nombre_Apellido = Nombre_Apellido;
}
public String getResidencia() {
    return Residencia;
}
public void setResidencia(String Residencia) {
    this.Residencia = Residencia;
}
}
```

#### 4.4.2 Clase Servicio

Esta clase se encarga de representar los servicios disponibles en la barbería, incluyendo descripciones, precios y duración estimada.

```
package com.mycompany.barberiacjev4.logica;
import java.io.Serializable;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.OneToOne;
/* Clase Servicio.
 * Representa un servicio ofrecido en la barbería. Utiliza anotaciones de JPA para mapear la información en la
 * base de datos y tiene una relación uno a uno con la clase Cliente.*/
@Entity
public class Servicio implements Serializable {
    // Identificador único para el servicio (clave primaria en la base de datos)
    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE)
    private int num_Servicio;
    private String Tipo_Cabello;
    private String Tipo_Corte;
    private String Sede;
    private String Barbero;
    private String Adicionales;
    private String Asesoría;
    // Relación uno a uno con la clase Cliente
    @OneToOne
    private Cliente unServicio;
    /*
     * Constructor por defecto. Necesario para las operaciones de JPA.
     */
    public Servicio() {
    }
    /*
     * Constructor con parámetros.
     * Permite inicializar un servicio con todos sus atributos.
     * @param num_Servicio Identificador único del servicio
     * @param Tipo_Cabello Tipo de cabello
     * @param Tipo_Corte Tipo de corte
     * @param Sede Sede de la barbería
     * @param Barbero Nombre del barbero
     * @param Adicionales Servicios adicionales
     * @param Asesoría Indicación de asesoría
     * @param unServicio Cliente asociado al servicio
     */
    public Servicio(int num_Servicio, String Tipo_Cabello, String Tipo_Corte, String Sede, String Barbero, String Adicionales, String Asesoría, Cliente unServicio) {
        this.num_Servicio = num_Servicio;
        this.Tipo_Cabello = Tipo_Cabello;
        this.Tipo_Corte = Tipo_Corte;
        this.Sede = Sede;
        this.Barbero = Barbero;
        this.Adicionales = Adicionales;
        this.Asesoría = Asesoría;
        this.unServicio = unServicio;
    }
}
```

Activate Windows

```

// Métodos getter y setter para cada atributo
/*Obtiene el identificador único del servicio.
 * @return num_Servicio*/
public int getNum_Servicio() {
    return num_Servicio;
}
public void setNum_Servicio(int num_Servicio) {
    this.num_Servicio = num_Servicio;
}
public String getTipo_Cabello() {
    return Tipo_Cabello;
}
public void setTipo_Cabello(String Tipo_Cabello) {
    this.Tipo_Cabello = Tipo_Cabello;
}
public String getTipo_Corte() {
    return Tipo_Corte;
}
public void setTipo_Corte(String Tipo_Corte) {
    this.Tipo_Corte = Tipo_Corte;
}
public String getSede() {
    return Sede;
}
public void setSede(String Sede) {
    this.Sede = Sede;
}
public String getBarbero() {
    return Barbero;
}
public void setBarbero(String Barbero) {
    this.Barbero = Barbero;
}
public String getAdicionales() {
    return Adicionales;
}
public void setAdicionales(String Adicionales) {
    this.Adicionales = Adicionales;
}
public String getAsesoria() {
    return Asesoria;
}
public void setAsesoria(String Asesoria) {
    this.Asesoria = Asesoria;
}
public Cliente getUnServicio() {
    return unServicio;
}
public void setUnServicio(Cliente unServicio) {
    this.unServicio = unServicio;
}
}

```

#### 4.4.3 Clase Controladora

Diseñada para manejar la comunicación entre las interfaces de usuario y el JPA

Controller, permitiendo realizar operaciones CRUD sobre la base de datos de manera eficiente.

```
package com.mycompany.barberiacjev4.logica;
import com.mycompany.barberiacjev4.persistencia.ControladoraPersistencia;
import java.util.List;
// Clase Controladora que actúa como puente entre la lógica de la aplicación y la persistencia de datos
public class Controladora {
    // Instancia de la clase ControladoraPersistencia para manejar las operaciones de base de datos
    ControladoraPersistencia controlPersis = new ControladoraPersistencia();
    // Método para guardar un cliente y un servicio asociado en la base de datos
    public void guardar(String cedula, String contacto, String corte, String nombre, String residencia, String adicionales, String asesoria, String barbero,
        String sede, String cabello) {
        // Creamos un cliente y asignamos sus valores a partir de los parámetros
        Cliente cliente = new Cliente();
        cliente.setCedula(cedula);
        cliente.setNombre_Apellido(nombre);
        cliente.setNum_Contacto(contacto);
        cliente.setResidencia(residencia);
        // Creamos el servicio y asignamos sus valores
        Servicio servicio = new Servicio();
        servicio.setAdicionales(adicionales);
        servicio.setAsesoria(asesoria);
        servicio.setBarbero(barbero);
        servicio.setSede(sede);
        servicio.setTipo_Cabello(cabello);
        servicio.setTipo_Corte(corte);
        // Asociamos el cliente al servicio
        servicio.setUnServicio(unservicio: cliente);
        // Guardamos el cliente y el servicio en la base de datos
        controlPersis.guardar(cliente, servicio);
    }
    // Método para traer todos los servicios desde la base de datos
    public List<Servicio> traerServicios() {
        // Llamamos al método de persistencia que retorna la lista de servicios
        return controlPersis.traerServicios();
    }
    // Método para borrar un servicio específico por su número
    public void borrarServicio(int num_servicio) {
        // Llamamos al método de persistencia para eliminar el servicio
        controlPersis.borrarServicio(num_servicio);
    }
    // Método para traer un servicio específico por su número
    public Servicio traerServicios(int num_servicio) {
        // Retornamos el servicio encontrado en la base de datos
        return controlPersis.traerServicios(num_servicio);
    }
}
```

```
// Método para modificar los datos de un servicio y el cliente asociado
public void modificarServicio(Servicio servicio, String adicionales, String asesoria, String barbero, String sede, String cabello, String residencia,
    String nombre, String corte, String contacto, String cedula) {
    // Actualizamos los valores del servicio
    servicio.setAdicionales(adicionales);
    servicio.setAsesoria(asesoria);
    servicio.setBarbero(barbero);
    servicio.setSede(sede);
    servicio.setTipo_Cabello(cabello);
    servicio.setTipo_Corte(corte);
    // Modificamos el servicio en la base de datos
    controlPersis.modificarServicio(servicio);
    // Buscamos al cliente asociado al servicio para actualizar su información
    Cliente cliente = this.buscarCliente(id_cliente: servicio.getUnServicio().getId_cliente());
    cliente.setCedula(cedula);
    cliente.setNombre_Apellido(nombre);
    cliente.setNum_Contacto(contacto);
    cliente.setResidencia(residencia);
    // Actualizamos los datos del cliente en la base de datos
    this.modificarCliente(cliente);
}
// Método privado para buscar un cliente por su ID
private Cliente buscarCliente(int id_cliente) {
    // Retornamos el cliente encontrado en la base de datos
    return controlPersis.traerCliente(id_cliente);
}
// Método privado para modificar los datos de un cliente en la base de datos
private void modificarCliente(Cliente cliente) {
    // Llamamos al método de persistencia para actualizar el cliente
    controlPersis.modificarCliente(cliente);
}
}
```

## 4.5 Interfaz de Usuario (Código)

Se desarrollaron los siguientes módulos principales:

### 4.5.1 Panel Principal

Desde aquí, los usuarios pueden navegar entre las diferentes funcionalidades de la aplicación.

```
package com.mycompany.barberiacjev4.igu;
/*
 * Clase Principal que extiende javax.swing.JFrame y actúa como la ventana principal de la aplicación.
 * Contiene botones para navegar a otras funcionalidades de la aplicación y un diseño gráfico personalizado.
 */
public class Principal extends javax.swing.JFrame {
    /*
     * Constructor de la clase Principal.
     * Inicializa los componentes gráficos mediante el método initComponents().
     */
    public Principal() {
        initComponents();
    }

    @SuppressWarnings("unchecked")
    Generated Code
    /*
     * Método invocado al presionar el botón "Registrar".
     * Abre una nueva ventana para ingresar datos.
     *
     * @param evt Evento de acción asociado al botón.
     */
    private void btnRegistrarActionPerformed(java.awt.event.ActionEvent evt) {
        IngresarDatos pantalla = new IngresarDatos();// Crea una nueva ventana de tipo IngresarDatos.
        pantalla.setVisible(b: true);// Hace visible la ventana "IngresarDatos".
        pantalla.setLocationRelativeTo(c: null);// Centra la ventana en la pantalla.
    }

    /*
     * Método invocado al presionar el botón "Ver Datos".
     * Abre una nueva ventana para visualizar datos registrados.
     *
     * @param evt Evento de acción asociado al botón.
     */
    private void btnDatosActionPerformed(java.awt.event.ActionEvent evt) {
        VerDatos pantalla = new VerDatos();// Crea una nueva ventana de tipo VerDatos.
        pantalla.setVisible(b: true);// Hace visible la ventana "VerDatos".
        pantalla.setLocationRelativeTo(c: null);// Centra la ventana en la pantalla.
    }

    // Variables declaration - do not modify
    private javax.swing.JButton btnDatos;
    private javax.swing.JButton btnRegistrar;
    private javax.swing.JButton jButton1;
    private javax.swing.JLabel jLabel1;
    private javax.swing.JPanel jPanel1;
    // End of variables declaration
}
```

### 4.5.2 Ingreso de Datos

Permite a los clientes registrar su información personal, seleccionar servicios y elegir la sede de preferencia.

```
package com.mycompany.barberiacjev4.igu;

import com.mycompany.barberiacjev4.logica.Controladora;
import javax.swing.JDialog;
import javax.swing.JOptionPane;
/*
 * Clase IngresarDatos.
 * Proporciona una interfaz gráfica para que los usuarios puedan registrar datos de clientes y servicios
 * en el sistema de la barbería.
 */
public class IngresarDatos extends javax.swing.JFrame {
    // Instancia de la controladora para manejar la lógica del sistema
    Controladora control = new Controladora();

    /*
     * Constructor de la clase IngresarDatos.
     * Inicializa los componentes de la interfaz.
     */
    public IngresarDatos() {
        initComponents();// Inicialización de los componentes gráficos
    }

    @SuppressWarnings("unchecked")
    Generated Code
    /*
     * Método para limpiar todos los campos de entrada de datos.
     * Resetea los valores de los campos y las listas desplegables.
     */
    private void btnLimpiarActionPerformed(java.awt.event.ActionEvent evt) {
        // Limpiar los campos de texto
        txtCedula.setText("");
        txtContacto.setText("");
        txtCorte.setText("");
        txtNombre.setText("");
        txtResidencia.setText("");
        // Restablecer las listas desplegables a su valor inicial
        cmbAdicionales.setSelectedIndex(anIndex:0);
        cmbAsesoria.setSelectedIndex(anIndex:0);
        cmbBarbero.setSelectedIndex(anIndex:0);
        cmbSede.setSelectedIndex(anIndex:0);
        cmbCabello.setSelectedIndex(anIndex:0);
    }
}
```



```

/*
 * Método para guardar los datos ingresados por el usuario.
 * Los datos se recogen de los campos de entrada y se envían a la controladora
 * para ser almacenados en la base de datos.
 */
private void btnGuardarActionPerformed(java.awt.event.ActionEvent evt) {
    // Recopilar los datos del cliente
    String cedula = txtCedula.getText();
    String contacto = txtContacto.getText();
    String corte = txtCorte.getText();
    String nombre = txtNombre.getText();
    String residencia = txtResidencia.getText();
    // Recopilar los datos del servicio
    String adicionales = (String) cmbAdicionales.getSelectedItem();
    String asesoria = (String) cmbAsesoria.getSelectedItem();
    String barbero = (String) cmbBarbero.getSelectedItem();
    String sede = (String) cmbSede.getSelectedItem();
    String cabello = (String) cmbCabello.getSelectedItem();
    // Llamar al método guardar de la controladora para registrar los datos
    control.guardar(cedula, contacto, corte, nombre, residencia, adicionales, asesoria, barbero, sede, cabello);
    // Mostrar un mensaje de confirmación al usuario
    JOptionPane optionPanel = new JOptionPane(message: "Se ha Guardado con Exito el Formulario", messageType: JOptionPane.INFORMATION_MESSAGE);
    JDialog dialog = optionPanel.createDialog(title: "Guardado Exitoso");
    dialog.setAlwaysOnTop(alwaysOnTop: true);
    dialog.setVisible(true);
}

```

```

}
// Variables declaration - do not modify
private javax.swing.JLabel Celular;
private javax.swing.JLabel Celular1;
private javax.swing.JButton btnGuardar;
private javax.swing.JButton btnLimpiar;
private javax.swing.JComboBox<String> cmbAdicionales;
private javax.swing.JComboBox<String> cmbAsesoria;
private javax.swing.JComboBox<String> cmbBarbero;
private javax.swing.JComboBox<String> cmbCabello;
private javax.swing.JComboBox<String> cmbSede;
private javax.swing.JLabel jLabel1;
private javax.swing.JLabel jLabel10;
private javax.swing.JLabel jLabel11;
private javax.swing.JLabel jLabel12;
private javax.swing.JLabel jLabel13;
private javax.swing.JLabel jLabel2;
private javax.swing.JLabel jLabel3;
private javax.swing.JLabel jLabel4;
private javax.swing.JLabel jLabel6;
private javax.swing.JLabel jLabel7;
private javax.swing.JLabel jLabel8;
private javax.swing.JLabel jLabel9;
private javax.swing.JPanel jPanel1;
private javax.swing.JPanel jPanel2;
private javax.swing.JTextField txtCedula;
private javax.swing.JTextField txtContacto;
private javax.swing.JTextField txtCorte;
private javax.swing.JTextField txtNombre;
private javax.swing.JTextField txtResidencia;
// End of variables declaration

```

### 4.5.3 Edición de Datos

Proporciona la funcionalidad de actualizar la información existente de los clientes y servicios.

```
package com.mycompany.barberiacjev4.igu;

import com.mycompany.barberiacjev4.logica.Controladora;
import com.mycompany.barberiacjev4.logica.Servicio;
import javax.swing.JDialog;
import javax.swing.JOptionPane;
/*
 * Clase EditarDatos.
 * Permite editar la información de un servicio y el cliente asociado. Esta interfaz de usuario
 * proporciona campos para modificar los datos almacenados en la base de datos.
 */
public class EditarDatos extends javax.swing.JFrame {
    // Instancia de la controladora para manejar la lógica del sistema
    Controladora control = null;
    // Identificador del servicio a editar
    int num_servicio;
    // Objeto servicio que contiene los datos del servicio seleccionado
    Servicio servicio;

    /*
     * Constructor de la clase EditarDatos.
     * Inicializa los componentes de la interfaz y carga los datos del servicio a editar.
     *
     * @param num_servicio Identificador del servicio a editar
     */
    public EditarDatos(int num_servicio) {
        control = new Controladora(); // Inicialización de la controladora
        //this.num_servicio = num_servicio;
        initComponents(); // Inicialización de los componentes gráficos
        IngresarDatos(num_servicio); // Carga los datos en la interfaz
    }

    @SuppressWarnings("unchecked")
    Generated Code
    /*
     * Método para limpiar todos los campos de la interfaz.
     * Resetea los valores de los campos de entrada y las listas desplegables.
     */
    private void btnLimpiarActionPerformed(java.awt.event.ActionEvent evt) {

        txtCedula.setText("");
        txtContacto.setText("");
        txtCorte.setText("");
        txtNombre.setText("");
        txtResidencia.setText("");
        cmbAdicionales.setSelectedIndex(anIndex:0);
        cmbAsesoria.setSelectedIndex(anIndex:0);
        cmbBarbero.setSelectedIndex(anIndex:0);
        cmbSede.setSelectedIndex(anIndex:0);
        cmbCabello.setSelectedIndex(anIndex:0);
    }
}
```

```

/*
 * Método para guardar los cambios realizados en los datos del servicio y cliente.
 * Recoge los datos de los campos y los actualiza en la base de datos.
 */
private void btnGuardarActionPerformed(java.awt.event.ActionEvent evt) {

    // Recopila los datos del cliente
    String cedula = txtCedula.getText();
    String contacto = txtContacto.getText();
    String corte = txtCorte.getText();
    String nombre = txtNombre.getText();
    String residencia = txtResidencia.getText();

    // Modifica el servicio y el cliente asociado
    String adicionales = (String) cmbAdicionales.getSelectedItem();
    String asesoria = (String) cmbAsesoria.getSelectedItem();
    String barbero = (String) cmbBarbero.getSelectedItem();
    String sede = (String) cmbSede.getSelectedItem();
    String cabello = (String) cmbCabello.getSelectedItem();

    control.modificarServicio(servicio, adicionales, asesoria, barbero, sede, cabello, residencia, nombre, corte, contacto, cedula);

    // Muestra un mensaje de confirmación
    mostrarMensaje(mensaje: "Edición realizada correctamente", tipo: "Info", titulo: "Edición Correcta");
    // Redirige a la ventana de visualización de datos
    VerDatos pantalla = new VerDatos();
    pantalla.setVisible(b: true);
    pantalla.setLocationRelativeTo(c: null);

    this.dispose();// Cierra la ventana actual
}

```

```

// Variables declaration - do not modify
private javax.swing.JLabel Celular;
private javax.swing.JLabel Celular1;
private javax.swing.JButton btnGuardar;
private javax.swing.JButton btnLimpiar;
private javax.swing.JComboBox<String> cmbAdicionales;
private javax.swing.JComboBox<String> cmbAsesoria;
private javax.swing.JComboBox<String> cmbBarbero;
private javax.swing.JComboBox<String> cmbCabello;
private javax.swing.JComboBox<String> cmbSede;
private javax.swing.JLabel jLabel1;
private javax.swing.JLabel jLabel10;
private javax.swing.JLabel jLabel11;
private javax.swing.JLabel jLabel12;
private javax.swing.JLabel jLabel13;
private javax.swing.JLabel jLabel2;
private javax.swing.JLabel jLabel3;
private javax.swing.JLabel jLabel4;
private javax.swing.JLabel jLabel6;
private javax.swing.JLabel jLabel7;
private javax.swing.JLabel jLabel8;
private javax.swing.JLabel jLabel9;
private javax.swing.JPanel jPanel1;
private javax.swing.JPanel jPanel2;
private javax.swing.JTextField txtCedula;
private javax.swing.JTextField txtContacto;
private javax.swing.JTextField txtCorte;
private javax.swing.JTextField txtNombre;
private javax.swing.JTextField txtResidencia;
// End of variables declaration

```

```
// End of variables declaration
/*
 * Método para cargar los datos del servicio en los campos de la interfaz.
 * Ajusta los valores según los datos obtenidos del servicio seleccionado.
 *
 * @param num_servicio Identificador del servicio a editar
 */
private void IngresarDatos(int num_servicio) {
    this.servicio = control.traerServicios(num_servicio);
    // Rellena los campos con los datos existentes
    txtCedula.setText(t: servicio.getUnServicio().getCedula());
    txtContacto.setText(t: servicio.getUnServicio().getNum_Contacto());
    txtCorte.setText(t: servicio.getTipo_Corte());
    txtNombre.setText(t: servicio.getUnServicio().getNombre_Apellido());
    txtResidencia.setText(t: servicio.getUnServicio().getResidencia());

    //editar tipo de cabello
    if (servicio.getTipo_Cabello().equals(anObject: "Liso")) {
        cmbCabello.setSelectedIndex(anIndex:1);
    } else if (servicio.getTipo_Cabello().equals(anObject: "Ondulado")) {
        cmbCabello.setSelectedIndex(anIndex:2);
    } else if (servicio.getTipo_Cabello().equals(anObject: "Rizado")) {
        cmbCabello.setSelectedIndex(anIndex:3);
    } else if (servicio.getTipo_Cabello().equals(anObject: "Afro")) {
        cmbCabello.setSelectedIndex(anIndex:4);
    }
    //editar Sede
    if (servicio.getSede().equals(anObject: "Chapinero")) {
        cmbSede.setSelectedIndex(anIndex:1);
    } else if (servicio.getSede().equals(anObject: "Candelaria")) {
        cmbSede.setSelectedIndex(anIndex:2);
    } else if (servicio.getSede().equals(anObject: "Suba")) {
        cmbSede.setSelectedIndex(anIndex:3);
    } else if (servicio.getSede().equals(anObject: "Bosa")) {
        cmbSede.setSelectedIndex(anIndex:4);
    }
    //editar Barbero
    if (servicio.getBarbero().equals(anObject: "Valentina")) {
        cmbBarbero.setSelectedIndex(anIndex:1);
    } else if (servicio.getBarbero().equals(anObject: "Cesar")) {
        cmbBarbero.setSelectedIndex(anIndex:2);
    } else if (servicio.getBarbero().equals(anObject: "Jhonny")) {
        cmbBarbero.setSelectedIndex(anIndex:3);
    } else if (servicio.getBarbero().equals(anObject: "Eliut")) {
        cmbBarbero.setSelectedIndex(anIndex:4);
    }
}
```

```

//editar Adicionales
if (servicio.getAdicionales().equals(anObject: "Corte de cejas")) {
    cmbAdicionales.setSelectedIndex(anIndex:1);
} else if (servicio.getAdicionales().equals(anObject: "Corte Barba")) {
    cmbAdicionales.setSelectedIndex(anIndex:2);
} else if (servicio.getAdicionales().equals(anObject: "Tinturado")) {
    cmbAdicionales.setSelectedIndex(anIndex:3);
}
//editar Asesoria
if (servicio.getAsesoria().equals(anObject: "Si")) {
    cmbAsesoria.setSelectedIndex(anIndex:1);
} else if (servicio.getAsesoria().equals(anObject: "No")) {
    cmbAsesoria.setSelectedIndex(anIndex:2);
}
}
/*
 * Muestra un mensaje emergente en la pantalla.
 *
 * @param mensaje Texto del mensaje
 * @param tipo Tipo de mensaje ("Info", "Error", etc.)
 * @param titulo Título del mensaje
 */
public void mostrarMensaje(String mensaje, String tipo, String titulo){

    int messageType;
    if (tipo.equals(anObject: "Info")) {
        messageType = JOptionPane.INFORMATION_MESSAGE;
    } else if (tipo.equals(anObject: "Error")) {
        messageType = JOptionPane.ERROR_MESSAGE;
    } else {
        messageType = JOptionPane.PLAIN_MESSAGE;
    }

    JOptionPane optionPanel = new JOptionPane(message:mensaje, messageType);
    JDialog dialog = optionPanel.createDialog(title: titulo);
    dialog.setAlwaysOnTop(alwaysOnTop:true);
    dialog.setVisible(b: true);

}
}

```

```
// Variables declaration - do not modify
private javax.swing.JButton btnEditar;
private javax.swing.JButton btnEliminar1;
private javax.swing.JLabel jLabel1;
private javax.swing.JLabel jLabel2;
private javax.swing.JLabel jLabel5;
private javax.swing.JPanel jPanel1;
private javax.swing.JPanel jPanel2;
private javax.swing.JScrollPane jScrollPane1;
private javax.swing.JTable tablaServicio;
// End of Variables declaration
// End of Variables declaration

private void cargarTabla() {
    //definimos el modelo que vamos a querer para la tabla
    DefaultTableModel modeloTabla = new DefaultTableModel() {
        //esto hace que las filas y las columnas no sean editables
        @Override
        public boolean isCellEditable(int row, int column) {
            return false;
        }
    };
    //{"Num Servicio", "Cliente", "Cedula", "Contacto", "Residencia", "Sede", "Barbero", "Tipo Cabello", "Tipo de Corte", "Adicionales", "Asesoría"}
    //establecemos los nombres de la columna
    String titulos[] = {"Num Servicio", "Cliente", "Cedula", "Contacto", "Residencia", "Sede", "Barbero", "Tipo Cabello", "Tipo de Corte", "Adicionales", "Asesoría"};
    modeloTabla.setColumnIdentifiers(titulos);
    //cargar los datos desde la base de datos
    List<Servicio> listaServicios = control.traservicios();
    //recorrer la lista y mostrar cada uno de los elementos en la tabla
    if(!listaServicios.isEmpty()){
        for(Servicio servicio : listaServicios) {
            Object[] objeto = {servicio.getNum_Servicio(), servicio.getUnServicio().getNombre_Apellido(), servicio.getUnServicio().getCedula(),
                servicio.getUnServicio().getNum_Contacto(),
                servicio.getUnServicio().getResidencia(), servicio.getSede(), servicio.getBarbero(), servicio.getTipo_Cabello(),
                servicio.getTipo_Corte(), servicio.getAdicionales(), servicio.getAsesoría()};/*{servicio.getNum_Servicio(), servicio.getSede(),
                servicio.getBarbero(), servicio.getTipo_Cabello(),
                servicio.getTipo_Corte(), servicio.getAdicionales(), servicio.getAsesoría(),
                servicio.getUnServicio().getNombre_Apellido(), servicio.getUnServicio().getCedula(), servicio.getUnServicio().getNum_Contacto(),
                servicio.getUnServicio().getResidencia()};*/
            modeloTabla.addRow(objeto);
        }
    }
    tablaServicio.setModel(modeloTabla);
}
}
```

#### 4.5.4 Consulta de Datos

Facilita la visualización de información almacenada, tanto para el personal administrativo como para los clientes.

```
package com.mycompany.barberiacjev4.igu;
import com.mycompany.barberiacjev4.logica.Controladora;
import com.mycompany.barberiacjev4.logica.Servicio;
import java.util.List;
import javax.swing.JOptionPane;
import javax.swing.table.DefaultTableModel;
public class VerDatos extends javax.swing.JFrame {
    Controladora control = null;
    public VerDatos() {
        control = new Controladora();
        initComponents();
    }
    @SuppressWarnings("unchecked")
    // Generated Code
    private void formWindowOpened(java.awt.event.WindowEvent evt) {
        cargarTabla();
    }
    private void btnEditarActionPerformed(java.awt.event.ActionEvent evt) {
        //control que la tabla no este vacia
        if (tablaServicio.getRowCount() > 0) {
            //control que se halla se haya seleccionado un servicio o linea
            if (tablaServicio.getSelectedRow() != -1) {
                //obtengo la id del servicio que voy a editar
                int num_servicio = Integer.parseInt(s: String.valueOf(tablaServicio.getValueAt(row: tablaServicio.getSelectedRow(), column: 0)));
                EditarDatos pantallaEdit = new EditarDatos(num_servicio);
                pantallaEdit.setVisible(true);
                pantallaEdit.setLocationRelativeTo(this);
                this.dispose();
            }
            else{
                mostrarMensaje(mensaje:"No selecciono ningun servicio", tipo:"Error", titulo: "Error al editar");
            }
        }
        else{
            mostrarMensaje(mensaje:"No hay nada para editar en la Tabla", tipo:"Error", titulo: "Error al editar");
        }
    }
}
```

```

private void btnEliminarActionPerformed(java.awt.event.ActionEvent evt) {
    //controlo que la tabla no este vacia
    if (tablaServicio.getRowCount() > 0) {
        //controlo que se halla se haya seleccionado un servicio o linea
        if (tablaServicio.getSelectedRow() != -1) {
            //obtengo la id del servicio que voy a eliminar
            int num_servicio = Integer.parseInt(s: String.valueOf(obj:tablaServicio.getValueAt(row:tablaServicio.getSelectedRow(), column: 0)));
            //Llamo al metodo borrar
            control.borrarServicio(num_servicio);
            //aviso al cliente que borro correctamente
            mostrarMensaje(mensaje:"Servicio eliminado correctamente", tipo: "Info", titulo: "Borrado de Servicio");
            //se actualiza la tabla en la interfaz grafica
            cargarTabla();
        }
        else{
            mostrarMensaje(mensaje:"No selecciono ningun servicio", tipo:"Error", titulo: "Error al eliminar");
        }
    }
    else{
        mostrarMensaje(mensaje:"No hay nada para eliminar en la Tabla", tipo: "Error", titulo: "Error al eliminar");
    }
}

public void mostrarMensaje(String mensaje, String tipo, String titulo){
    int messageType;
    if (tipo.equals(anObject: "Info")) {
        messageType = JOptionPane.INFORMATION_MESSAGE;
    } else if (tipo.equals(anObject: "Error")) {
        messageType = JOptionPane.ERROR_MESSAGE;
    } else {
        messageType = JOptionPane.PLAIN_MESSAGE;
    }
    JOptionPane optionPanel = new JOptionPane(mensaje:mensaje, messageType);
    JDialog dialog = optionPanel.createDialog(title: titulo);
    dialog.setAlwaysOnTop(alwaysOnTop:true);
    dialog.setVisible(b: true);
}

```

```

// Variables declaration - do not modify
private javax.swing.JButton btnEditar;
private javax.swing.JButton btnEliminar1;
private javax.swing.JLabel jLabel1;
private javax.swing.JLabel jLabel2;
private javax.swing.JLabel jLabel5;
private javax.swing.JPanel jPanel1;
private javax.swing.JPanel jPanel2;
private javax.swing.JScrollPane jScrollPane1;
private javax.swing.JTable tablaServicio;
// End of variables declaration
private void cargarTabla() {
    //definimos el modelo que vamos a querer para la tabla
    DefaultTableModel modeloTabla = new DefaultTableModel() {
        //esto hace que las filas y las columnas no sean editables
        @Override
        public boolean isCellEditable (int row, int column) {
            return false;
        }
    };
    //{"Num Servicio", "Cliente", "Cedula", "Contacto", "Residencia", "Sede", "Barbero", "Tipo Cabello", "Tipo de Corte", "Adicionales", "Asesoría"}
    //establecemos los nombres de la columnas
    String titulos[] = {"Num Servicio", "Cliente", "Cedula", "Contacto", "Residencia", "Sede", "Barbero", "Tipo Cabello", "Tipo de Corte", "Adicionales", "Asesoría"};
    modeloTabla.setColumnIdentifiers(titulos);
    //cargar los datos desde la base de datos
    List<Servicio> listaServicios = control.traservicios();
    //recorrer la lista y mostrar cada uno de los elemntos en la tabla
    if(listaServicios!=null){
        for(Servicio servicio : listaServicios) {
            Object[] objeto = {servicio.getNum_Servicio(), servicio.getUnServicio().getNombre_Apellido(), servicio.getUnServicio().getCedula(),
                servicio.getUnServicio().getNum_Contacto(),
                servicio.getUnServicio().getResidencia(), servicio.getSede(), servicio.getBarbero(), servicio.getTipo_Cabello(),
                servicio.getTipo_Corte(), servicio.getAdicionales(), servicio.getAsesoría()};/*(servicio.getNum_Servicio(), servicio.getSede(),
                servicio.getBarbero(), servicio.getTipo_Cabello(),
                servicio.getTipo_Corte(), servicio.getAdicionales(), servicio.getAsesoría(),
                servicio.getUnServicio().getNombre_Apellido(), servicio.getUnServicio().getCedula(), servicio.getUnServicio().getNum_Contacto(),
                servicio.getUnServicio().getResidencia());*/
            modeloTabla.addRow(-rowData:objeto);
        }
    }
    tablaServicio.setModel(dataModel: modeloTabla);
}

```

La integración con **JPA Controller** aseguró la conexión eficiente y segura entre la aplicación y la base de datos. Este componente maneja las transacciones y operaciones **CRUD**, permitiendo la persistencia de los datos de manera confiable.

Finalmente, la aplicación fue sometida a un proceso exhaustivo de pruebas. Estas incluyeron validación de datos, pruebas de carga y simulaciones de escenarios reales de uso, garantizando su robustez y usabilidad.

Gracias a este proyecto, "**Barberiacjev**" cuenta ahora con una herramienta tecnológica avanzada que optimiza la gestión de clientes y servicios, mejorando significativamente la experiencia del usuario y la eficiencia operativa de la barbería.

## 5 Conclusiones

El proyecto "Barberiacjev" fue una experiencia enriquecedora que nos permitió aplicar y reforzar conocimientos adquiridos durante el curso de Programación Avanzada. A través de herramientas como Java, Apache Maven y JPA Controller, desarrollamos una aplicación que centraliza la gestión de clientes y servicios en la barbería, optimizando tanto la experiencia del usuario como la organización interna. Nos enfocamos en un diseño modular que facilita el mantenimiento y escalabilidad del sistema, y en la implementación de funcionalidades clave como el registro, consulta y edición de datos, utilizando interfaces gráficas intuitivas y conectadas de manera eficiente a una base de datos en Wampserver64. Este proyecto nos retó a implementar conceptos complejos de manera práctica, desde el manejo de dependencias hasta operaciones CRUD, y nos dejó como resultado una solución tecnológica funcional, bien estructurada y lista para ser ampliada según las necesidades de la barbería.



## **6 Aspectos a mejorar**

Creemos que nuestra aplicación "Barberiacjev" puede mejorar en algunos aspectos importantes para hacerla más funcional y práctica. Primero, nos dimos cuenta de que el botón de salir en la ventana principal no cumple con su propósito, ya que no cierra la aplicación cuando se presiona. Esto se podría solucionar añadiendo el código necesario para que cierre correctamente todas las ventanas activas y termine la ejecución del programa.

Además, sería ideal mejorar la interacción entre las ventanas. Por ejemplo, al completar un registro de datos en la ventana de "Ingresar Datos", esta debería cerrarse automáticamente y redirigir al usuario a la ventana principal en lugar de quedarse abierta. Lo mismo aplica para otras ventanas, como la de "Editar Datos", donde al guardar los cambios se debería volver automáticamente al menú principal. Estas modificaciones harían que el flujo de la aplicación sea más fluido y fácil de usar, mejorando la experiencia para los usuarios.

## 7 Referencias

(2025). Oracle.com.

[https://education.oracle.com/java/java/pFamily\\_48?intcmp=WWOU11042424MPP055C002](https://education.oracle.com/java/java/pFamily_48?intcmp=WWOU11042424MPP055C002)

Porter, B., Zyl, J. van, & Lamy, O. (2025). Welcome to Apache Maven – Maven.

Apache.org. <https://maven.apache.org/>

WebSphere Application Server Network Deployment Liberty. (2024, January 30).

Ibm.com. <https://www.ibm.com/docs/es/was-liberty/nd?topic=liberty-java-persistence-api-jpa>

Programación modular y creciente necesidad de desarrollo de software seguro -

Lazarus Alliance, Inc. (2024). Lazarusalliance.com. [https://lazarusalliance.com/es/modular-](https://lazarusalliance.com/es/modular-programming-and-increasing-need-for-secure-software-development/#:~:text=La%20programaci%C3%B3n%20modular%20es%20una,que%20ahorra%20tiempo%20y%20recursos.)

[programming-and-increasing-need-for-secure-software-](https://lazarusalliance.com/es/modular-programming-and-increasing-need-for-secure-software-development/#:~:text=La%20programaci%C3%B3n%20modular%20es%20una,que%20ahorra%20tiempo%20y%20recursos.)

[development/#:~:text=La%20programaci%C3%B3n%20modular%20es%20una,que%20ahorra](https://lazarusalliance.com/es/modular-programming-and-increasing-need-for-secure-software-development/#:~:text=La%20programaci%C3%B3n%20modular%20es%20una,que%20ahorra%20tiempo%20y%20recursos.)

[%20tiempo%20y%20recursos.](https://lazarusalliance.com/es/modular-programming-and-increasing-need-for-secure-software-development/#:~:text=La%20programaci%C3%B3n%20modular%20es%20una,que%20ahorra%20tiempo%20y%20recursos.)

